

SmartCare

Hospital Management System

Database Technical Report

Version 2.0 | PostgreSQL 15+ | April 2026

Software Engineering | Helwan University | Team 2

16

Tables

9

Triggers

3

Roles

2

EXCLUDE

Table of Contents

- 1. Database Overview
- 2. ENUM Types & Custom Types
- 3. Table Definitions (16 tables)
 - 3.1 Identity & Roles — *app_user, patient, doctor*
 - 3.2 Clinic & Scheduling — *clinic, clinic_reservation, appointment*
 - 3.3 Clinical Records — *medicalrecord, prescription, drug, drug_details, prescription_details*
 - 3.4 Billing — *invoice, payment*
 - 3.5 Notifications & Reporting — *notification, notificationpreference, report*
- 4. Relationships
- 5. Constraints
- 6. Indexes
- 7. Triggers

1. Database Overview

SmartCare is a hospital management system designed to handle patient registration, appointment scheduling, clinical record management, pharmacy prescriptions, clinic room reservations, billing, notifications, and administrative reporting. The database is implemented in PostgreSQL 15+ and consists of 16 tables organised into five functional domains.

Tables	Triggers	Enum Types	Custom Types	EXCLUDE Constraints	Indexes
16	9	8	1	2	18

Domain Groups

Domain	Tables	Purpose
Identity & Roles	app_user, patient, doctor	Manages all user accounts and role-based profiles.
Clinic & Scheduling	clinic, clinic_reservation, appointment	Controls room availability and appointment booking.
Clinical Records	medicalrecord, prescription, drug, drug_details, prescription_details	Tracks diagnoses, prescriptions, and drug catalogues.
Billing	invoice, payment	Handles invoice generation and payment settlement.
Notifications & Reporting	notification, notificationpreference, report	Delivers system messages and admin analytics.

2. ENUM Types & Custom Types

PostgreSQL does not support inline ENUM definitions. All enumerated types are declared before any table creation. A custom range type timerange is also declared for time-overlap checking in clinic_reservation.

Type Name	Values	Purpose
role_enum	Admin, Doctor, Patient	User role — drives all access control
appt_status_enum	pending, confirmed, cancelled, completed	Appointment lifecycle status
notif_type_enum	appointment, prescription, billing, system_error	Notification trigger category
channel_enum	email, in_app	Notification delivery channel
inv_status_enum	pending, paid, refunded	Invoice payment status
pay_method_enum	card, digital_wallet	Payment method
report_type_enum	doctor_performance, appointment_trends, billing_summary	Type of analytics report
export_fmt_enum	PDF, Excel	Report export format
timerange	Custom range type over TIME	Used by EXCLUDE constraints on clinic_reservation

3. Table Definitions

3.1 Identity & Roles

app_user

TABLE

Central identity table for every system user. Stores login credentials, role assignment, and account status. All other role-specific tables (patient, doctor) extend this table via a 1:1 relationship. Also serves as the direct FK target for appointment, medicalrecord, clinic_reservation, notification, notificationpreference, and report — replacing separate patient/doctor FKs. The role column is the system's single source of truth for access control.

Column	Type	Key	Nullabl e	Description
user_id	INT IDENTITY	PK	NO	Auto-generated primary key
name	VARCHAR(100)		NO	Full name of the user
email	VARCHAR(150)		NO	Login email — must be unique across all users
password_hash	VARCHAR(255)		NO	Bcrypt-hashed password — never stored in plain text
role	role_enum		NO	Admin / Doctor / Patient — drives all access control
is_active	BOOLEAN		NO	Soft-enable flag — false disables login without deleting
created_at	TIMESTAMP		NO	Account creation timestamp — default NOW()
phone_number	VARCHAR(20)		YES	Contact number — optional

patient

TABLE

Extends app_user for Patient-role accounts. Stores personal medical details that are irrelevant to doctors and admins: date of birth, gender, home address, and blood type. The user_id FK links back to app_user in a strict 1:1 total participation — a patient profile cannot exist without a user account, and every Patient-role user must have exactly one patient profile. A trigger enforces that user_id references a Patient-role user.

Column	Type	Key	Nullabl e	Description
patient_id	INT IDENTITY	PK	NO	Auto-generated primary key
user_id	INT	FK	NO	FK → app_user(user_id) — 1:1, UNIQUE, NOT NULL
date_of_birth	DATE		NO	Patient date of birth
gender	VARCHAR(10)		NO	Gender
address	TEXT		YES	Residential address
blood_type	VARCHAR(5)		YES	Blood type e.g. A+, O-

doctor

TABLE

Extends `app_user` for Doctor-role accounts. Stores professional credentials: medical specialty and license number. The `license_number` carries a `UNIQUE` constraint to prevent duplicate registrations. Like `patient`, this table enforces a 1:1 total participation with `app_user` via a trigger that blocks non-Doctor-role users from receiving a doctor profile.

Column	Type	Key	Nullabl e	Description
doctor_id	INT IDENTITY	PK	NO	Auto-generated primary key
user_id	INT	FK	NO	FK → app_user(user_id) — 1:1, UNIQUE, NOT NULL
specialty	VARCHAR(100)		NO	Medical specialty e.g. Cardiology
license_number	VARCHAR(50)		NO	Medical license number — UNIQUE

3.2 Clinic & Scheduling

clinic

TABLE

Represents a physical clinic room or unit within the hospital. The `type` column categorises the clinic (e.g. outpatient, emergency, ICU). Clinics are the slots that doctors reserve via `clinic_reservation`. The table is intentionally minimal — it acts as a reference entity that gives clinic reservations a meaningful anchor beyond a simple room number.

Column	Type	Key	Nullabl e	Description
clinic_id	INT IDENTITY	PK	NO	Auto-generated primary key
type	VARCHAR(100)		NO	Clinic type e.g. outpatient, emergency, ICU

clinic_reservation

TABLE

Records the time slots during which a doctor works in a specific clinic on a given day. Uses `TIME` columns for `start_hour` and `end_hour` to represent hours of the day precisely. Two `EXCLUDE` constraints (backed by a custom `timerange` type and the `btree_gist` extension) prevent any time overlap — both within the same clinic (no two doctors at the same time in the same room) and for the same doctor (cannot be in two clinics simultaneously). A `CHECK` constraint ensures `end_hour > start_hour`. A trigger enforces that `doctor_id` references a Doctor-role user.

Column	Type	Key	Nullabl e	Description
reservation_id	INT IDENTITY	PK	NO	Auto-generated primary key
clinic_id	INT	FK	NO	FK → clinic(clinic_id)
doctor_id	INT	FK	NO	FK → app_user(user_id) — must be Doctor role
day	DATE		NO	The calendar date of this reservation
start_hour	TIME		NO	Shift start time e.g. 09:00
end_hour	TIME		NO	Shift end time e.g. 13:00 — must be > start_hour

appointment

TABLE

Records a scheduled visit between a patient and a doctor at a specific date and time. Both `patient_id` and `doctor_id` reference `app_user(user_id)` directly — the `role` column in `app_user` distinguishes who is who. A trigger (`trg_appointment_roles`) validates that `patient_id` belongs to a Patient-role user and `doctor_id` belongs to a Doctor-role user on every `INSERT` and `UPDATE`. A `UNIQUE` constraint on (`doctor_id`, `date_time`) prevents double-booking. A separate trigger blocks appointments booked in the past. The `status` column drives downstream logic: only a completed appointment can generate an invoice.

Column	Type	Key	Nullabl e	Description
<code>appointment_id</code>	INT IDENTITY	PK	NO	Auto-generated primary key
<code>patient_id</code>	INT	FK	NO	FK → <code>app_user(user_id)</code> — must be Patient role
<code>doctor_id</code>	INT	FK	NO	FK → <code>app_user(user_id)</code> — must be Doctor role
<code>date_time</code>	TIMESTAMP		NO	Date and time of the appointment — must be future
<code>status</code>	<code>appt_status_enum</code>		NO	pending / confirmed / cancelled / completed
<code>notes</code>	TEXT		YES	Optional appointment notes

3.3 Clinical Records

medicalrecord

TABLE

Clinical record created after a visit. Contains the doctor's diagnosis, clinical notes, and an optional link to an uploaded document. Both `patient_id` and `doctor_id` reference `app_user(user_id)` directly — a trigger enforces correct roles. The `appointment_id` FK is nullable (Partial:Partial) — a record can exist without a linked appointment for walk-in or emergency cases. When the linked appointment is deleted, `appointment_id` is set to `NULL` via `ON DELETE SET NULL` rather than cascading the delete.

Column	Type	Key	Nullabl e	Description
<code>record_id</code>	INT IDENTITY	PK	NO	Auto-generated primary key
<code>patient_id</code>	INT	FK	NO	FK → <code>app_user(user_id)</code> — must be Patient role
<code>doctor_id</code>	INT	FK	NO	FK → <code>app_user(user_id)</code> — must be Doctor role
<code>appointment_id</code>	INT	FK	YES	FK → <code>appointment(appointment_id)</code> — nullable
<code>visit_date</code>	DATE		NO	Date of the visit
<code>diagnosis</code>	TEXT		NO	Diagnosis details written by the doctor
<code>notes</code>	TEXT		YES	Additional clinical notes
<code>attached_document_url</code>	TEXT		YES	URL to any uploaded report or scan

prescription

TABLE

Header record for a digital prescription issued within a medical visit. Intentionally minimal — it acts as the parent record linking a medical record to one or more drug lines in `prescription_details`. This split design (header + detail lines) allows a single prescription to contain multiple drugs, each with its own dosage, frequency, and duration. The `issue_date` records when the prescription was written.

Column	Type	Key	Nullabl e	Description
<code>prescription_id</code>	INT IDENTITY	PK	NO	Auto-generated primary key
<code>record_id</code>	INT	FK	NO	FK → medicalrecord(record_id)
<code>issue_date</code>	DATE		NO	Date the prescription was issued

drug

TABLE

Master catalogue of drugs available in the system. Each drug has a name and is referenced by `prescription_details` when a doctor prescribes it. Separating drugs into their own table allows a drug to be referenced across many prescriptions without duplicating its name, and enables the `drug_details` table to store ingredient information independently of any specific prescription.

Column	Type	Key	Nullabl e	Description
<code>drug_id</code>	INT IDENTITY	PK	NO	Auto-generated primary key
<code>drug_name</code>	VARCHAR(150)		NO	Name of the drug e.g. Amoxicillin

drug_details

TABLE

Stores the active ingredients of each drug. A drug can have multiple active ingredients — for example, a combination antibiotic may contain two compounds. This is a 1:N relationship from `drug` to `drug_details`. The table is a reference catalogue maintained by administrators and is separate from prescription data — it describes what a drug contains, not how it is prescribed.

Column	Type	Key	Nullabl e	Description
<code>ingredient_id</code>	INT IDENTITY	PK	NO	Auto-generated primary key
<code>drug_id</code>	INT	FK	NO	FK → drug(drug_id)
<code>active_ingredient</code>	VARCHAR(150)		NO	Name of the active ingredient

prescription_details

TABLE

One row per drug line within a prescription. Stores the dosage, frequency, and duration for a specific drug as prescribed in a specific prescription. This table is the junction between prescription and drug — it carries the clinical specifics of how each drug is to be taken. A single prescription can have many detail lines (one per drug prescribed), and the same drug can appear in many prescription detail lines across different prescriptions.

Column	Type	Key	Nullabl e	Description
prescription_details_id	INT IDENTITY	PK	NO	Auto-generated primary key
prescription_id	INT	FK	NO	FK → prescription(prescription_id)
drug_id	INT	FK	NO	FK → drug(drug_id)
dosage	VARCHAR(50)		NO	Drug dosage e.g. 500mg
frequency	VARCHAR(50)		NO	Administration frequency e.g. twice daily
duration	VARCHAR(50)		YES	Treatment duration e.g. 7 days

3.4 Billing

invoice

TABLE

Auto-generated billing record created when an appointment is marked completed. A trigger (trg_invoice_completed_only) enforces that no invoice can be created for a pending, confirmed, or cancelled appointment. The UNIQUE constraint on appointment_id enforces the strict 1:1 relationship — one appointment produces exactly one invoice. The status tracks the payment lifecycle: pending → paid (set automatically by trg_invoice_mark_paid when a payment row is inserted) → refunded.

Column	Type	Key	Nullabl e	Description
invoice_id	INT IDENTITY	PK	NO	Auto-generated primary key
appointment_id	INT	FK	NO	FK → appointment(appointment_id) — UNIQUE
issue_date	DATE		NO	Date the invoice was issued
total_amount	NUMERIC(10,2)		NO	Total amount due — CHECK >= 0
status	inv_status_enum		NO	pending / paid / refunded

payment

TABLE

Records a settled invoice. Created by the system when the patient completes a payment. The **UNIQUE** constraint on `invoice_id` enforces the 1:1 relationship — one invoice has at most one payment record. A **CHECK** constraint ensures `amount > 0`. Inserting a payment row automatically triggers `trg_invoice_mark_paid` which flips the linked invoice status to paid. The `receipt_pdf_url` stores a link to the generated PDF receipt.

Column	Type	Key	Nullabl e	Description
<code>payment_id</code>	INT IDENTITY	PK	NO	Auto-generated primary key
<code>invoice_id</code>	INT	FK	NO	FK → invoice(<code>invoice_id</code>) — UNIQUE
<code>payment_date</code>	DATE		NO	Date the payment was made
<code>amount</code>	NUMERIC(10,2)		NO	Amount paid — CHECK > 0
<code>method</code>	pay_method_enum		NO	card / digital_wallet
<code>receipt_pdf_url</code>	TEXT		YES	URL of the generated PDF receipt

3.5 Notifications & Reporting

notification

TABLE

System messages delivered to users via email or in-app channels. Every notification must belong to a user (total on notification side) but a user may have received no notifications yet (partial on user side). The `type` column categorises what triggered the notification (appointment change, prescription issued, billing event, or system error). The `is_read` flag defaults to false and is flipped when the user reads it.

Column	Type	Key	Nullabl e	Description
<code>notification_id</code>	INT IDENTITY	PK	NO	Auto-generated primary key
<code>user_id</code>	INT	FK	NO	FK → app_user(<code>user_id</code>)
<code>type</code>	notif_type_enum		NO	appointment / prescription / billing / system_error
<code>channel</code>	channel_enum		NO	email / in_app
<code>message</code>	TEXT		NO	Notification message body
<code>sent_at</code>	TIMESTAMP		NO	Timestamp sent — default NOW()
<code>is_read</code>	BOOLEAN		NO	Read flag — default false

notificationpreference

TABLE

Stores each user's custom settings for which notification channels and event types are active. A **UNIQUE** constraint on (*user_id*, *channel*, *event_type*) prevents duplicate preference rows. If a user has no rows here, the system falls back to defaults. The *is_enabled* flag lets users toggle specific notification types on or off without deleting the preference row.

Column	Type	Key	Nullabl e	Description
preference_id	INT IDENTITY	PK	NO	Auto-generated primary key
user_id	INT	FK	NO	FK → app_user(user_id)
channel	channel_enum		NO	email / in_app
event_type	VARCHAR(50)		NO	Event that triggers the notification
is_enabled	BOOLEAN		NO	Enabled flag — default true

report

TABLE

Analytics report generated by an Admin-role user. A trigger (*trg_report_admin_role*) enforces that *admin_id* references an Admin-role user — doctors and patients cannot generate reports. The *report_type* **ENUM** defines what the report covers. The *export_format* determines whether the output is a PDF or an Excel file, with *file_url* storing the link to the generated file.

Column	Type	Key	Nullabl e	Description
report_id	INT IDENTITY	PK	NO	Auto-generated primary key
admin_id	INT	FK	NO	FK → app_user(user_id) — must be Admin role
report_type	report_type_enum		NO	doctor_performance / appointment_trends / billing_summary
generated_at	TIMESTAMP		NO	Generation timestamp — default NOW()
export_format	export_fmt_enum		NO	PDF / Excel
file_url	TEXT		YES	URL of the exported report file

4. Relationships

All 18 relationships are listed below. Participation notation: Total = every instance must participate (NOT NULL FK), Partial = participation is optional (nullable FK or no FK required).

#	From	Relationship	To	Cardinality	Participation	FK Column
1	app_user	extends	patient	1:1	Total:Total	user_id → app_user
2	app_user	extends	doctor	1:1	Total:Total	user_id → app_user
3	app_user	books (patient)	appointment	1:N	Total:Total	patient_id → app_user
4	app_user	assigned (doctor)	appointment	1:N	Total:Partial	doctor_id → app_user
5	app_user	has (patient)	medicalrecord	1:N	Total:Total	patient_id → app_user
6	app_user	writes (doctor)	medicalrecord	1:N	Partial:Total	doctor_id → app_user
7	appointment	produces	medicalrecord	1:1	Partial:Partial	appointment_id → appointment
8	medicalrecord	contains	prescription	1:N	Partial:Total	record_id → medicalrecord
9	prescription	has lines	prescription_details	1:N	Partial:Total	prescription_id → prescription
10	drug	used in	prescription_details	1:N	Partial:Total	drug_id → drug
11	drug	has ingredients	drug_details	1:N	Partial:Total	drug_id → drug
12	clinic	hosts	clinic_reservation	1:N	Partial:Total	clinic_id → clinic
13	app_user	reserves (doctor)	clinic_reservation	1:N	Partial:Total	doctor_id → app_user
14	appointment	generates	invoice	1:1	Total:Total	appointment_id → appointment
15	invoice	settled by	payment	1:1	Partial:Partial	invoice_id → invoice
16	app_user	receives	notification	1:N	Partial:Total	user_id → app_user
17	app_user	configures	notificationreference	1:N	Partial:Partial	user_id → app_user
18	app_user	generates	report	1:N	Partial:Partial	admin_id → app_user

5. Constraints

All non-FK constraints. Foreign keys are implied by the relationship table above. EXCLUDE constraints use the btree_gist extension and the custom timerange type.

Type	Table	Name	On	Purpose
UNIQUE	app_user	uq_user_email	email	Login email must be globally unique
UNIQUE	patient	uq_patient_user	user_id	Enforces 1:1 with app_user
UNIQUE	doctor	uq_doctor_user	user_id	Enforces 1:1 with app_user
UNIQUE	doctor	uq_doctor_license	license_number	No duplicate medical licenses
UNIQUE	appointment	uq_doctor_timeslot	doctor_id, date_time	Prevents double-booking a doctor
UNIQUE	invoice	uq_invoice_appointment	appointment_id	Enforces 1:1 with appointment
UNIQUE	payment	uq_payment_invoice	invoice_id	Enforces 1:1 with invoice
UNIQUE	notificationpreference	uq_notifpref	user_id, channel, event_type	No duplicate preference rows
CHECK	clinic_reservation	chk_clinic_hours	end_hour > start_hour	Shift must end after it starts
CHECK	invoice	chk_invoice_amount	total_amount >= 0	Invoice cannot be negative
CHECK	payment	chk_payment_amount	amount > 0	Payment must be positive
EXCLUDE	clinic_reservation	no_clinic_overlap	clinic_id, day, timerange(..)	No two doctors in same clinic at same time
EXCLUDE	clinic_reservation	no_doctor_overlap	doctor_id, day, timerange(..)	Doctor cannot be in two clinics simultaneously

6. Indexes

18 explicit indexes covering the most frequent query patterns. UNIQUE and EXCLUDE constraints also auto-create indexes — those are listed in the constraints section and not repeated here.

Index Name	Table	Columns	Covers
idx_user_role	app_user	role	List all users by role (Admin/Doctor/Patient)
idx_user_active	app_user	is_active	Filter active accounts on login
idx_reservation_clinic_day	clinic_reservation	clinic_id, day	Check clinic availability on a given day
idx_reservation_doctor_day	clinic_reservation	doctor_id, day	Check a doctor's schedule on a given day
idx_appointment_patient	appointment	patient_id, date_time	Patient's upcoming appointments ordered by date
idx_appointment_doctor	appointment	doctor_id, date_time	Doctor's schedule + double-booking check
idx_appointment_status	appointment	status	Admin dashboard filter by appointment status
idx_appointment_date	appointment	date_time	Date-range scheduling views
idx_mr_patient	medicalrecord	patient_id, visit_date	Patient history ordered by date
idx_mr_doctor	medicalrecord	doctor_id, visit_date	Records written by a specific doctor
idx_mr_appointment	medicalrecord	appointment_id	JOIN from appointment — missing FK index fix
idx_prescription_record	prescription	record_id	All prescriptions within a medical record
idx_pd_prescription	prescription_details	prescription_id	Drug lines for a given prescription
idx_pd_drug	prescription_details	drug_id	All prescriptions that use a specific drug
idx_drug_details_drug	drug_details	drug_id	Active ingredients of a drug
idx_invoice_status	invoice	status	Billing dashboard — filter unpaid invoices
idx_notification_user	notification	user_id, is_read, sent_at	Unread notifications per user ordered by time
idx_report_admin	report	admin_id, generated_at	Reports generated by an admin ordered by date

7. Triggers

9 triggers enforce business rules that cannot be expressed with FK or CHECK constraints alone. Each trigger calls a dedicated PL/pgSQL function. Role-check triggers are the primary guard ensuring referential integrity across the flat user model — since all `patient_id`, `doctor_id`, and `admin_id` columns point directly to `app_user(user_id)`, only the role column distinguishes them.

1. `trg_patient_role_check`

BEFORE INSERT

ON patient

Ensures the linked `user_id` belongs to a Patient-role user. Raises an exception if a non-Patient user is used.

2. `trg_doctor_role_check`

BEFORE INSERT

ON doctor

Ensures the linked `user_id` belongs to a Doctor-role user. Raises an exception if a non-Doctor user is used.

3. `trg_appointment_roles`

BEFORE INSERT/UPDATE

ON appointment

Validates that `patient_id` references a Patient-role user AND `doctor_id` references a Doctor-role user in the same check.

4. `trg_mr_roles`

BEFORE INSERT/UPDATE

ON medicalrecord

Validates that `patient_id` references a Patient-role user AND `doctor_id` references a Doctor-role user.

5. `trg_reservation_doctor_role`

BEFORE INSERT/UPDATE

ON clinic_reservation

Ensures `doctor_id` references a Doctor-role user before allowing a clinic reservation.

6. `trg_invoice_completed_only`

BEFORE INSERT

ON invoice

Blocks invoice creation unless the linked appointment has `status = completed`.

7. `trg_invoice_mark_paid`

AFTER INSERT

ON payment

Automatically sets `invoice.status = paid` when a payment row is successfully inserted.

8. `trg_appointment_future_only`

BEFORE INSERT

ON appointment

Rejects any appointment where `date_time <= NOW()`, preventing past bookings.

9. `trg_report_admin_role`

BEFORE INSERT

ON report

Ensures `admin_id` references an Admin-role user. Blocks doctors and patients from generating reports.